

constexpr *INVOKE*

Document #: P1065R1
Date: 2019-06-16
Project: Programming Language C++
LWG
Reply-to: Tomasz Kamiński
<tomaszkam@gmail.com>
Barry Revzin
<barry.revzin@gmail.com>

Contents

[1 Revision History](#)
[2 Motivation](#)
[3 History](#)
[4 Proposal](#)
 [4.1 Wording](#)
[5 Acknowledgements](#)
[6 References](#)

§ 1 Revision History

Since [[P1065R0](#)], just wording changes to correctly describe what it means for things bind to be `constexpr` and also including `bind_front` `()`.

§ 2 Motivation

Currently, one of the most important utility functions in the standard library,

`std::invoke` `()`, is not `constexpr`. Even though
`std::apply` `()` and `std::visit` `()`, both of which rely on
INVOKE, are both `constexpr`. The standard library thus finds itself in an odd state where
`std::invoke` `()` is and is not `constexpr`.

The reason that `std::invoke` `()` is not `constexpr` has some interesting history associated with it. But at this point, it is simply history, and there is no further blocker to making this change. This proposal resolves [[LWG2894](#)] but also goes one step further and addresses various other *INVOKE*-related machinery.

§ 3 History

Our tale begins in April, 2015 with [\[llvm23141\]](#), which presented this code which broke in clang in C++14 (but had compiled in C++11 mode) due to the introduction of a `constexpr __invoke` (which ended up breaking range-v3):

```
#include <functional>
#include <type_traits>

struct Fun
{
    template<typename T, typename U>
    void operator()(T && t, U && u) const
    {
        static_assert(std::is_same<U, int &>::value, "");
    }
};

int main()
{
    std::bind(Fun{}, std::placeholders::_1, 42)("hello");
}
```

as well as the similar [\[llvm23135\]](#), which was about this program:

```
template<typename T>
int f(T x)
{
    return x.get();
}

template<typename T>
constexpr int g(T x)
{
    return x.get();
}

int main() {

    // O.K. The body of `f` is not required.
    decltype(f(0)) a;

    // Seems to instantiate the body of `g`
    // and results in an error.
    decltype(g(0)) b;

    return 0;
}
```

In both cases the fundamental issue was eager instantiation of the body, which doesn't actually seem necessary to determine the results here. In neither example is the return type deduced.

These are incarnations of [\[CWG1581\]](#), which dealt with the question of when, exactly, are `constexpr` functions defined. In the broken programs above, the `constexpr` functions (the non-`const` call operator of the binder object being returned in the first case and

g `()` in the second) were eagerly instantiated, triggering hard compile errors, in cases where the program ultimately would not have required their instantiation.

Thankfully, this difficult problem has been resolved by the adoption of [\[P0859R0\]](#) in Albuquerque, 2017. As a result, both of the above programs are valid.

This issue was the blocker for having a `constexpr std::invoke()` due to this eager instantiation issue - which no longer exists.

§ 4 Proposal

This proposal adds `constexpr` to the following *INVOKE*-related machinery: `invoke()`, `reference_wrapper<T>, not_fn()`, `bind()`, `bind_front()`, and `mem_fn()`. The remaining non-`constexpr` elements of the library that are *INVOKE*-adjacent are `function<Sig>`, `packaged_task<Sig>`, `async()`, `thread`, and `call_once()`.

This proposal resolves [\[LWG2894\]](#), [\[LWG2957\]](#), and [\[LWG3023\]](#). The last is addressed by guaranteeing that call wrappers that are produced by `not_fn()` and `bind()` have the same type if their state entities have the same type (note that this guarantee does not imply any restriction on implementors). Thus the types of `f1`, `f2`, `f3`, and `f4` in the following example are now guaranteed to be the same:

```
auto func = [](std::string) {};  
std::string s("foo");  
auto f1 = std::bind(func, s);  
auto f2 = std::bind(std::as_const(func), std::as_const(s));  
auto f3 = std::bind(func, std::string("bar"));  
auto f4 = std::bind(std::move(func), std::move(s));
```

The wording uses the phrase “shall be `constexpr` functions” in a couple places. We don’t seem to have a way to say that in Library, see also [\[LWG2833\]](#) and [\[LWG2289\]](#).

§ 4.1 Wording

Add `constexpr` to several places in the synopsis in 20.14.1 [functional.syn]

```
namespace std {  
    // [func.invoke], invoke  
    template<class F, class... Args>  
    - invoke_result_t<F, Args...> invoke(F&& f, Args&&... args)  
    + constexpr invoke_result_t<F, Args...> invoke(F&& f, Args&&...  
args)  
        noexcept(is_nothrow_invocable_v<F, Args...>);  
  
    // [refwrap], reference_wrapper  
    template<class T> class reference_wrapper;  
  
    - template<class T> reference_wrapper<T> ref(T&) noexcept;  
    - template<class T> reference_wrapper<const T> cref(const T&)
```

```

noexcept;
+ template<class T> constexpr reference_wrapper<T> ref(T&) noexcept;
+ template<class T> constexpr reference_wrapper<const T> cref(const
T&) noexcept;
    template<class T> void ref(const T&&) = delete;
    template<class T> void cref(const T&&) = delete;

- template<class T> reference_wrapper<T> ref(reference_wrapper<T>)
noexcept;
- template<class T> reference_wrapper<const T>
cref(reference_wrapper<T>);
+ template<class T> constexpr reference_wrapper<T>
ref(reference_wrapper<T>) noexcept;
+ template<class T> constexpr reference_wrapper<const T>
cref(reference_wrapper<T>) noexcept;

    // [arithmetic.operations], arithmetic operations
    // ...

    // [comparisons], comparisons
    // ...

    // [logical.operations], logical operations
    // ...

    // [bitwise.operations], bitwise operations
    // ...

    // [func.identity], identity
    // ...

    // [func.not.fn], function template not_fn
- template<class F> unspecified not_fn(F&& f);
+ template<class F> constexpr unspecified not_fn(F&& f);

    // [func.bind.front], function template bind_front
- template<class F, class... Args> unspecified bind_front(F&&,
Args&&...);
+ template<class F, class... Args> constexpr unspecified
bind_front(F&&, Args&&...);

    // [func.bind], bind
    template<class T> struct is_bind_expression;
    template<class T> struct is_placeholder;

- template<class F, class... BoundArgs>
-   unspecified bind(F&&, BoundArgs&&...);
- template<class R, class F, class... BoundArgs>
-   unspecified bind(F&&, BoundArgs&&...);
+ template<class F, class... BoundArgs>
+   constexpr unspecified bind(F&&, BoundArgs&&...);
+ template<class R, class F, class... BoundArgs>
+   constexpr unspecified bind(F&&, BoundArgs&&...);

    namespace placeholders {

```

```

        // M is the implementation-defined number of placeholders
        see below _1;
        see below _2;
        .
        .
        .
        see below _M;
    }

    // [func.memfn], member function adaptors
    template<class R, class T>
-   unspecified mem_fn(R T::*) noexcept;
+   constexpr unspecified mem_fn(R T::*) noexcept;

    // ...
}

```

The definition of the *simple call wrapper* (used only for `mem_fn`) is changed to be a refinement of *perfect forwarding call wrapper*, instead of *argument forwarding call wrapper*. These make the invocation operator conditionally `constexpr` and `noexcept`. In addition we state explicitly the copy/move constructor/assignment of simple call wrapper is core constant expression. [*Note*: The definition of simple call wrapper is still required to guarantee assignability.]

The requirement of copy/move operation to be defined in terms of state entities is now extended to any argument forwarding call wrapper (as we define them for `not_fn` and `bind`).

Apply following changes to 20.14.3 [func.require]:

- 3 Every call wrapper ([func.def]) **meets the `is_Cpp17MoveConstructible` and `Cpp17Destructible` requirements.** ~~A An argument forwarding call wrapper is a call wrapper that can be called with an arbitrary argument list and delivers the arguments to the wrapped callable object as references. This forwarding step delivers rvalue arguments as rvalue references and lvalue arguments as lvalue references. A simple call wrapper is an argument forwarding call wrapper that is `Cpp17CopyConstructible` and `Cpp17CopyAssignable` and whose copy constructor, move constructor, copy assignment operator, and move assignment operator do not throw exceptions.~~ [*Note*: In a typical implementation, argument forwarding call wrappers have an overloaded function call operator of the form

```

    template<class... UnBoundArgs>
-   R operator()(UnBoundArgs&&... unbound_args) cv-qual;
+   constexpr R operator()(UnBoundArgs&&... unbound_args) cv-qual;

```

—end note]

- 4 A *perfect forwarding call wrapper* is an argument forwarding call wrapper that forwards its state entities to the underlying call expression. This forwarding step delivers a state entity of type `T` as `cv T` & when the call is performed on an lvalue of the call wrapper type and as `cv T` && otherwise, where `cv` represents the cv-qualifiers of the call wrapper and where `cv` shall be neither `volatile` nor `const volatile`.

- 5 A *call pattern* defines the semantics of invoking a perfect forwarding call wrapper. A postfix call performed on a perfect forwarding call wrapper is expression-equivalent ([defns.expression-equivalent]) to an expression *e* determined from its call pattern *cp* by replacing all occurrences of the arguments of the call wrapper and its state entities with references as described in the corresponding forwarding steps.
- a A simple call wrapper is a perfect forwarding call wrapper that meets the Cpp17CopyConstructible and Cpp17CopyAssignable and whose copy constructor, move constructor, and assignment operator are constexpr functions which do not throw exceptions.
- 6 The copy/move constructor of a perfect argument forwarding call wrapper has the same apparent semantics as if memberwise copy/move of its state entities were performed ([class.copy.ctor]). [Note: This implies that each of the copy/move constructors has the same exception-specification as the corresponding implicit definition and is declared as constexpr if the corresponding implicit definition would be considered to be constexpr. —end note]
- 7 Perfect Argument forwarding call wrappers returned by a given standard library function template have the same type if the types of their corresponding state entities are the same.

Add constexpr to std ::invoke () in 20.14.4 [func.invoke]

```
template<class F, class... Args>
- invoke_result_t<F, Args...> invoke(F&& f, Args&&... args)
+ constexpr invoke_result_t<F, Args...> invoke(F&& f, Args&&...
args)
    noexcept(is_nothrow_invocable_v<F, Args...>);
```

Add constexpr to std ::reference_wrapper <T > in 20.14.5 [refwrap]

```
namespace std {
    template<class T> class reference_wrapper {
    public:
        // types
        using type = T;

        // construct/copy/destroy
        template<class U>
        - reference_wrapper(U&&) noexcept(see below);
        - reference_wrapper(const reference_wrapper& x) noexcept;
        + constexpr reference_wrapper(U&&) noexcept(see below);
        + constexpr reference_wrapper(const reference_wrapper& x)
noexcept;

        // assignment
        - reference_wrapper& operator=(const reference_wrapper& x)
noexcept;
        + constexpr reference_wrapper& operator=(const reference_wrapper&
x) noexcept;

        // access
```

```

- operator T& () const noexcept;
- T& get() const noexcept;
+ constexpr operator T& () const noexcept;
+ constexpr T& get() const noexcept;

    // invocation
    template<class... ArgTypes>
- invoke_result_t<T&, ArgTypes...> operator()(ArgTypes&&...)
const;
+ constexpr invoke_result_t<T&, ArgTypes...> operator()
(ArgTypes&&...) const;
};
template<class T>
reference_wrapper(T&) -> reference_wrapper<T>;
}

```

And its corresponding subsections, 20.14.5.1 [refwrap.const]

```

template<class U>
- reference_wrapper(U&& u) noexcept(see below);
+ constexpr reference_wrapper(U&& u) noexcept(see below);

[... ]

- reference_wrapper(const reference_wrapper& x) noexcept;
+ constexpr reference_wrapper(const reference_wrapper& x) noexcept;

```

20.14.5.2 [refwrap.assign]

```

- reference_wrapper& operator=(const reference_wrapper& x) noexcept;
+ constexpr reference_wrapper& operator=(const reference_wrapper& x)
noexcept;

```

20.14.5.3 [refwrap.access]

```

- operator T& () const noexcept;
+ constexpr operator T& () const noexcept;

[... ]

- T& get() const noexcept;
+ constexpr T& get() const noexcept;

```

20.14.5.4 [refwrap.invoke]

```

template<class... ArgTypes>
- invoke_result_t<T&, ArgTypes...>
+ constexpr invoke_result_t<T&, ArgTypes...>
operator()(ArgTypes&&... args) const;

```

and its helper functions, 20.14.5.5 [refwrap.helpers]

```

- template<class T> reference_wrapper<T> ref(T& t) noexcept;
+ template<class T> constexpr reference_wrapper<T> ref(T& t)
noexcept;

```

2 Returns: reference_wrapper<T>(t).

```
- template<class T> reference_wrapper<T> ref(reference_wrapper<T> t)
noexcept;
+ template<class T> constexpr reference_wrapper<T>
ref(reference_wrapper<T> t) noexcept;
```

3 Returns: ref(t).get().

```
- template<class T> reference_wrapper<const T> cref(const T& t)
noexcept;
+ template<class T> constexpr reference_wrapper<const T> cref(const
T& t) noexcept;
```

4 Returns: reference_wrapper<const T>(t).

```
- template<class T> reference_wrapper<const T>
cref(reference_wrapper<T> t) noexcept;
+ template<class T> constexpr reference_wrapper<const T>
cref(reference_wrapper<T> t) noexcept;
```

5 Returns: cref(t).get().

Add constexpr to std::not_fn() in 20.14.12 [func.not.fn]:

```
- template<class F> unspecified not_fn(F&& f);
+ template<class F> constexpr unspecified not_fn(F&& f);
```

Add constexpr to std::bind_front() in 20.14.13 [func.bind.front]:

```
template<class F, class... Args>
- unspecified bind_front(F&& f, Args&&... args);
+ constexpr unspecified bind_front(F&& f, Args&&... args);
```

Apply the following changes to std::bind() in 20.14.14.3 [func.bind.bind], merging bind and bind<R>:

1 In the text that follows:

(1.0) — g is a value of the result of a bind invocation,

(1.1) — FD is the type decay_t<F>,

— [1.2]{.pnum fd is ~~an lvalue of type FD constructed from~~
~~std::forward<E>(f)~~, a target object of g
([func.def]) of type FD direct-non-list-initialized with
std::forward<F>(f),

(1.3) — τ_i is the i th type in the template parameter pack BoundArgs,

(1.4) — TD_i is the type decay_t< $\tau_i,$

(1.5) — t_i is the i th argument in the function parameter pack bound_args,

- (1.6) — td_i is ~~an lvalue of type ID_i constructed from $std::forward<T_i>(t_i)$~~ ; a bound argument entity of g ([func.def]) of type TD_i direct-non-list-initialized with $std::forward<T_i>(t_i)$,
- (1.7) — U_j is the j th deduced type of the $UnBoundArgs$ ~~$\&\&\dots$~~ parameter of the argument forwarding call wrapper, and
- (1.8) — u_j is the j th argument associated with U_j .

```
template<class F, class... BoundArgs>
- unspecified bind(F&& f, BoundArgs&&... bound_args);
+ constexpr unspecified bind(F&& f, BoundArgs&&... bound_args);
+ template<class R, class F, class... BoundArgs>
+ constexpr unspecified bind(F&& f, BoundArgs&&... bound_args);
```

- 2 ~~Requires~~ **Mandates**: $is_constructible_v$ ~~$<FD, F$~~ ~~$>$~~ ~~shall be is~~ true. For each T_i in $BoundArgs$, $is_constructible_v<TD_i, T_i>$ ~~shall be is~~ true.

- 2a **Expects**: FD and each TD_i meets the of *Cpp17MoveConstructible* and *Cpp17Destructible* requirements. $INVOKE(fd, w_1, w_2, \dots, w_N)$ ([func.require]) ~~shall be is~~ a valid expression for some values $w_1, w_2, \dots, w_N$, where N has the value $sizeof$ ~~$bound_args$~~ . ~~The cv-qualifiers cv of the call wrapper g, as specified below, shall be neither~~ ~~volatile nor~~ ~~const~~ ~~volatile~~.

- 3 **Returns**: An argument forwarding call wrapper g ([func.require]). A program that attempts to invoke a volatile-qualified g is ill-formed. When g is not volatile-qualified, ~~The effect of invocation~~ $g(u_1, u_2, \dots, u_M)$ ~~shall be is~~ expression-equivalent ([defns.expression-equivalent]) to ~~$INVOKE(fd, std::forward<V_1>(v_1), std::forward<V_2>(v_2), \dots, std::forward<V_N>(v_N))$~~ $INVOKE(static_cast<V_{fd}>(v_{fd}), static_cast<V_1>(v_1), static_cast<V_2>(v_2), \dots, static_cast<V_N>(v_N))$ for the first overload, and $INVOKE<R>(static_cast<V_{fd}>(v_{fd}), static_cast<V_1>(v_1), static_cast<V_2>(v_2), \dots, static_cast<V_N>(v_N))$ for the second overload, where the values and types of the target argument v_{fd} and of the bound arguments $v_1, v_2, \dots, v_N$ are determined as specified below. ~~The copy constructor and move constructor of the argument forwarding call wrapper shall throw an exception if and only if the corresponding constructor of FD or of any of the types ID_i throws an exception.~~

- 4 **Throws**: ~~Nothing unless the construction of fd or of one of the values td_i throws an exception.~~ Any exception thrown by the initialization of the state entities of g .

- 5 **Remarks**: The return type shall satisfy the *Cpp17MoveConstructible* requirements. If all of FD and ID_i satisfy the *Cpp17CopyConstructible* requirements, then the return type shall satisfy the *Cpp17CopyConstructible* requirements. [Note: This implies that all of FD and ID_i are *Cpp17MoveConstructible*. — end note]

5a [Note: If all of FD and TD_i meet the requirements of *Cpp17CopyConstructible*, then the return type meets the requirements of *Cpp17CopyConstructible*. -end note]

```
template<class R, class F, class... BoundArgs>
  unspecified bind(F&& f, BoundArgs&&... bound_args);
```

6 ~~Requires: $\text{is_constructible_v} \leq \text{FD}, \text{F} \geq$ shall be `true`. For each TD_i in `BoundArgs`, $\text{is_constructible_v} \leq \text{TD}_i, \text{TD}_i \geq$ shall be `true`. `INVOKE(f, w1, w2, ..., wN)` ([func.require]) shall be a valid expression for some values $w_1, w_2, \dots, w_N$, where N has the value `sizeof (bound_args)`. The cv-qualifiers cv of the call wrapper g, as specified below, shall be neither `volatile` nor `const` `volatile`.~~

7 ~~Returns: An argument forwarding call wrapper g ([func.require]). The effect of `g(u1, u2, ..., uN)` shall be `INVOKE<R>(fd, std::forward<V1>(v1), std::forward<V2>(v2), ..., std::forward<VN>(vN))` where the values and types of the bound arguments $v_1, v_2, \dots, v_N$ are determined as specified below. The copy constructor and move constructor of the argument forwarding call wrapper shall throw an exception if and only if the corresponding constructor of FD or of any of the types TD_i throws an exception.~~

8 ~~Throws: Nothing unless the construction of fd or of one of the values TD_i throws an exception.~~

9 ~~Remarks: The return type shall satisfy the *Cpp17MoveConstructible* requirements. If all of FD and TD_i satisfy the *Cpp17CopyConstructible* requirements, then the return type shall satisfy the *Cpp17CopyConstructible* requirements. [Note: This implies that all of FD and TD_i are *Cpp17MoveConstructible*. —end note]~~

Define v_{fd} and add reference to the cv-qualifies in 20.14.14.3 [func.bind.bind]/10:

10 The values of the *bound arguments* $v_1, v_2, \dots, v_N$ and their corresponding types $V_1, V_2, \dots, V_N$ depend on the types TD_i derived from the call to bind and the cv-qualifiers cv of the call wrapper g as follows:

(10.1) — if TD_i is `reference_wrapper` `<T>`, [...]

(10.2) — if the value of `is_bind_expression_v<TDi>` is `true`, the argument is `static_cast<TDi cv &>(std::forward<Uj>(uj)...)` and its type V_i is `invoke_result_t<TDi cv &, Uj...>&&`;

(10.3) — if the value j of [...]

(10.4) — otherwise, [...]

11 The value of the *target argument* v_{fd} is `fd` and its corresponding type V_{fd} is `FD cv &`.

Add constant requirement to the placeholders in 20.14.14.4 [func.bind.place]/1:

- ¹ All placeholder types ~~meet the shall be~~ `Cpp17DefaultConstructible` and `Cpp17CopyConstructible` requirements, and their default constructors and copy/move constructors ~~are constexpr functions which do shall~~ not throw exceptions. It is implementation-defined whether placeholder types ~~meet the are~~ `Cpp17CopyAssignable` requirements, but if so, their ~~Cpp17CopyAssignable placeholders'~~ copy assignment operators ~~are constexpr functions which do shall~~ not throw exceptions.

Add `constexpr` to `std::mem_fn` in 20.14.15 [func.memfn]

```
- template<class R, class T> unspecified mem_fn(R T::* pm) noexcept;  
+ template<class R, class T> constexpr unspecified mem_fn(R T::* pm)  
noexcept;
```

- ¹ Returns: A simple call wrapper ~~fn such that the expression fn(t, a₂, ..., a_N) is equivalent to INVOKE(pm, t, a₂, ..., a_N) ([func.require]).~~ with call pattern `invoke(pmd, call_args ...)`, where `pmd` is the target object of `fn` of type `R T::*` direct-non-list-initialized with `pm`, and `call_args` is an argument pack used in a function call expression (`[expr.call]`) of `pm`.

§ 5 Acknowledgements

Thanks to Casey Carter and Agustín Bergé for going over the history of issues surrounding `constexpr` `invoke` and suggesting that this proposal be written. Thanks to Daniel Krügler, Tim Song and, Casey Carter for help on the wording.

§ 6 References

[CWG1581] Richard Smith. 2012. When are constexpr member functions defined?

<https://wg21.link/cwg1581>

[llvm23135] Gonzalo BG. 2015. [C++11/14] Body of constexpr function templates instantiated too eagerly in unevaluated operands.

https://bugs.llvm.org/show_bug.cgi?id=23135

[llvm23141] Eric Niebler. 2015. `std::bind` const-qualifying bound arguments captured by value when compiled as C++14.

https://bugs.llvm.org/show_bug.cgi?id=23141

[LWG2289] Daniel Krügler. constexpr guarantees of defaulted functions still insufficient.

<https://wg21.link/lwg2289>

[LWG2833] Richard Smith. Library needs to specify what it means when it declares a function constexpr.

<https://wg21.link/lwg2833>

[LWG2894] Great Britain. The function template `std::apply()` is required to be constexpr, but `std::invoke()` isn't.

<https://wg21.link/lwg2894>

[LWG2957] Tim Song. bind's specification doesn't apply the cv-qualification of the call wrapper to the callable object.

<https://wg21.link/lwg2957>

[LWG3023] Detlef Vollmann. Clarify unspecified call wrappers.

<https://wg21.link/lwg3023>

[P0859R0] Richard Smith. 2017. Core Issue 1581: When are constexpr member functions defined?

<https://wg21.link/p0859r0>

[P1065R0] Barry Revzin. 2018. constexpr INVOKE.

<https://wg21.link/p1065r0>